

Introduction to Git and GitHub

Workshop Lead: Eric Huang

Facilitator: Judah Zammit

September 23rd 2026

Outline for this workshop:

1. Intro to Git and Version Control Systems
2. Basic Git functions
3. Advanced Git functions
4. Working with GitHub
5. Miscellaneous Features
6. Conclusions

Hands-on Activities:

- Quick Review Polls
- Git/GitHub exercises

What you will need:

1. GitHub Desktop App and GitHub Account
2. Text editor
3. UNIX-based Command Line (time-allowing)

What you WILL learn

- Basic theory and features behind Git and GitHub
- How to manage local and remote repositories

What you will NOT learn

- How to code
- Data Analysis Pipelines



"FINAL".doc



FINAL.doc!



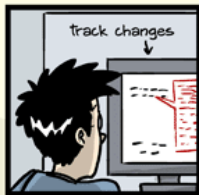
FINAL_rev.2.doc



FINAL_rev.6.COMMENTS.doc



FINAL_rev.8.comments5.
CORRECTIONS.doc



FINAL_rev.18.comments7.
corrections9.MORE.30.doc



FINAL_rev.22.comments49.
corrections.10.##\$%WHYDID
ICOMETOGRADSCHOOL?????.doc

JORGE CHAN © 2012

WWW.PHDCOMICS.COM



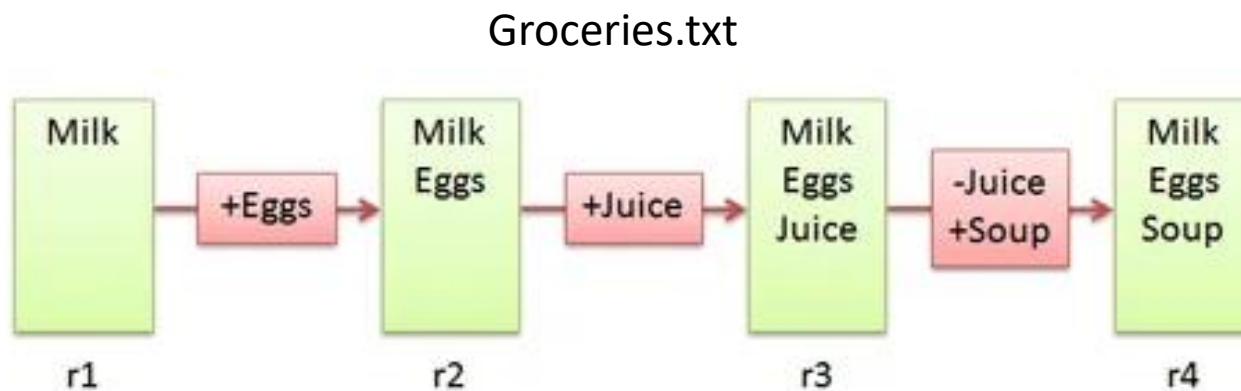
McGill

Quantitative Life
Sciences

Sciences quantitatives
du vivant

Version Control Systems

- Document changes
- Unlimited “undo” button
- Allow for multiple collaborators



Git

- A type of version control system
- Open-source
- The industry standard tool
- Designed for:
 - Speed
 - Simple design
 - Non-linear development (branching)
 - No centralized “truth”
 - Handle large projects efficiently





Git was created by **Linus Torvalds**, the same person who started the Linux kernel.



After only a few months of development, he handed the "maintainer" torch to **Junio Hamano**, a Japanese software engineer who had been a major contributor from the start.

What can Git do?

- Can track any file but works best for **pure text files** (.txt,.R,.py etc.)
 - Can track and display changes in pure text files
- Cannot display for binary files (e.g.: Word/PowerPoint)
 - Still useful for storing such files, just cannot use all of Git's features
- Markdown and LaTeX can be used instead of these binary files

```

docs/api/preprocessing.md
...
@@ -47,18 +47,24 @@ For visual quality control, see {func}`~scanpy.pl.highest_expr_genes` and
47 47     pp.recipe_seurat
48 48     ...
49 49
50 - ## Batch effect correction
50 + (pp-data-integration)=
51 51
52 - Also see {ref}`data-integration`. Note that a simple batch correction method is available via {func}`pp.regress_out`. Checkout {mod}`scanpy.external` for more.
52 + ## Data integration
53 +
54 + Batch effect correction and other data integration.
55 + Note that a simple batch correction method is available via {func}`pp.regress_out`.
53 56
54 57     ```{eval-rst}
55 58     .. autosummary::
56 59         :nosignatures:
57 60         :toctree: generated/
58 61
59 62     pp.combat
63 + pp.harmony_integrate
60 64     ...
61 65
66 + Also see {ref}`data integration tools <data-integration>` and external {ref}`external data integration <external-data-integration>`.
67 +
62 68     ## Doublet detection
63 69
64 70     ```{eval-rst}

```

Git

vs.

GitHub

- A version control system
 - Each project is a repository
- Works locally and remotely
- Is open-source

- A server that **hosts** Git repositories
 - Biggest host of open-source projects!
- A cloud-based service
- Owned by Microsoft (2018)



Terminology: Working directory vs. Git Repository

- Working directory is your **project folder**
- Git Repository is a **hidden folder** stored inside your working directory
 - Contains the necessary tracking of changes (Git history)



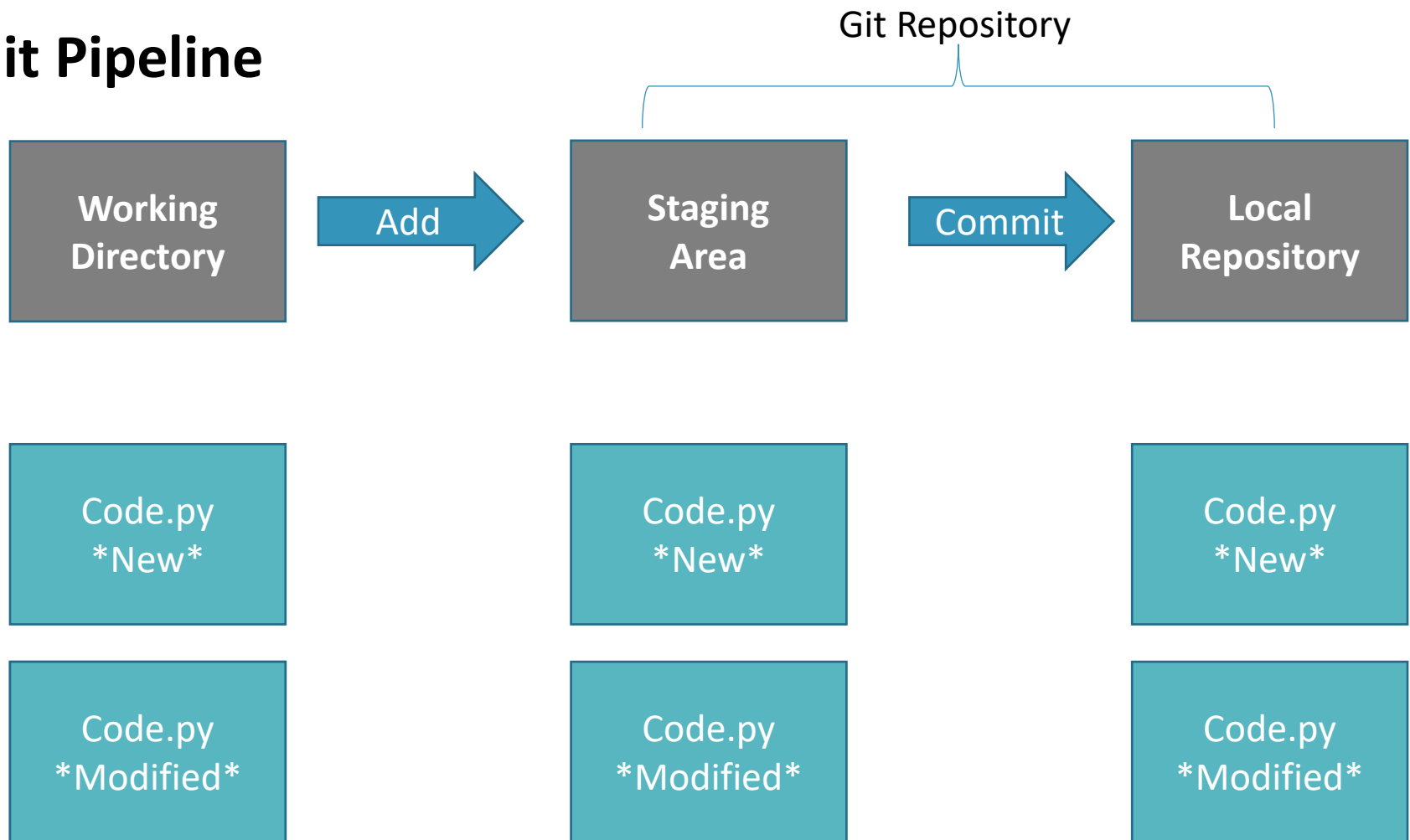
Terminology: Staging Area

- Rough draft area
- Track changes without adding it to history

Terminology: Commit

- Direct checkpoint to add to history
- Used when a change is complete

Git Pipeline



Commit Messages

- Every commit MUST have an attached message
- Identifies what changes have been made and why
- [filename added/changed ; what change ; why]

BAD:

```
git commit -m 'add user.rb'
```

GOOD:

```
git commit -m 'create user model to  
store user session information'
```

Common Commit Types

Type	When to Use It
feat	A new feature for the user, not a new feature for builds.
fix	A bug fix for the user, not a fix to a build script.
docs	Documentation only changes (e.g., editing a README).
style	Changes that do not affect the meaning of the code (white-space, formatting, missing semi-colons, etc).
refactor	A code change that neither fixes a bug nor adds a feature.
perf	A code change that improves performance.
test	Adding missing tests or correcting existing tests.
chore	Updating grunt tasks, build configurations, or local development tools; no production code change.



Poll #1

T/F – Both untracked and modified files must be **added** to the staging area before being committed

T/F - GitHub is a type of Version Control System

T/F – The repository keeps track of changes by storing a **new** version of each file at each commit

Summary

- Untracked or modified files are first **added** to the **staging area**
- Changes that have been staged can then be **committed** to the **local repository**
- Commits can then be used to **revert or keep track** of what changes have been made

Part II – Basic Git Features



Activity #0 – Initialize a Git Repository



Activity #0 – Initialize a Git Repository

1. Create a new Git Repository (File → new repo OR “create a new repository on your hard drive”)
2. Open the generated project folder, what do you see?



Activity #1 – Create and track a Text File with Git



Activity #1 – Create and track a Text File with Git

1. Create a text file (or code script) in the project folder
2. Add + commit the file to track it.
3. Look at your commit history to see your commit!
4. Modify the file and commit the changes. Try and make informative commit messages!

Activity #2 – Revert a file to a previous commit



Activity #2 – Revert a file to a previous commit

1. Make an unwanted change to your file
2. Use your staging area to discard the changes
3. Make another unwanted change and commit it
4. Now use your commit history to revert the commit!

Poll #2

T/F – After initialisation, a project folder becomes a git repository

T/F – After making a commit, you cannot revert to a previous version of your repository

Is it possible for two commits to have the same ID?



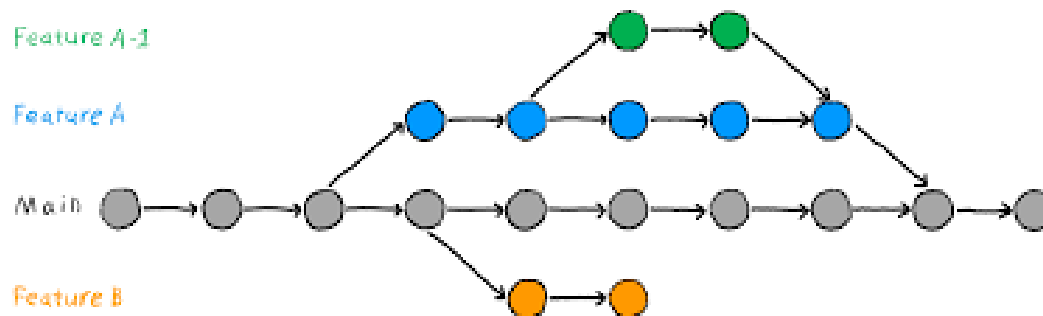
Part III – Advanced Git Features



What if you want to test out a new feature without affecting your current version of the code?

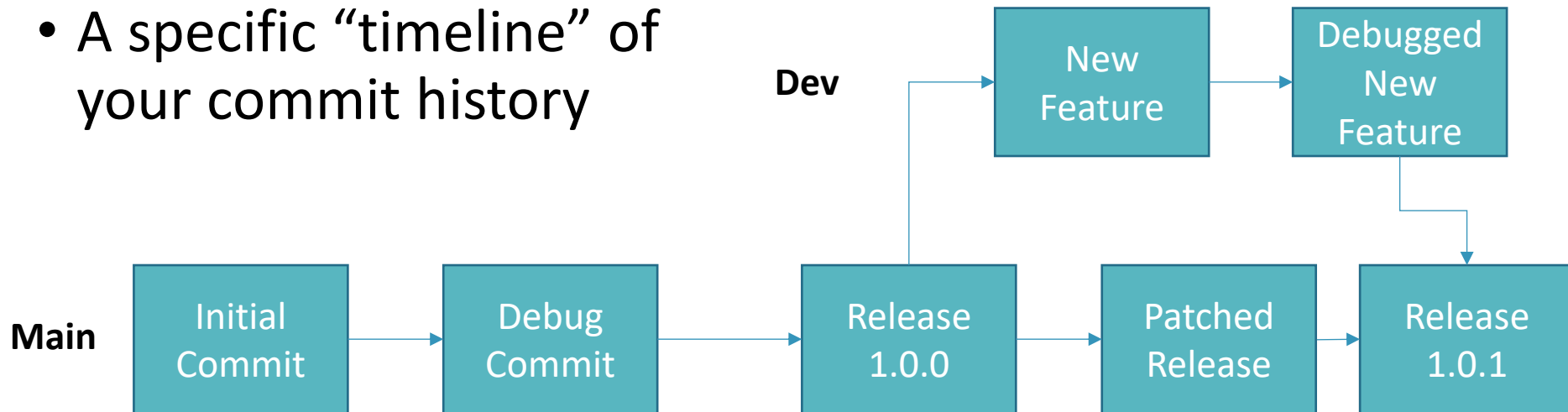
What if you want to collaborate with others on separate features?

Branching



What is a Branch?

- A specific “timeline” of your commit history



- Implement new features without disrupting the release code

Activity #3 – Create a new branch

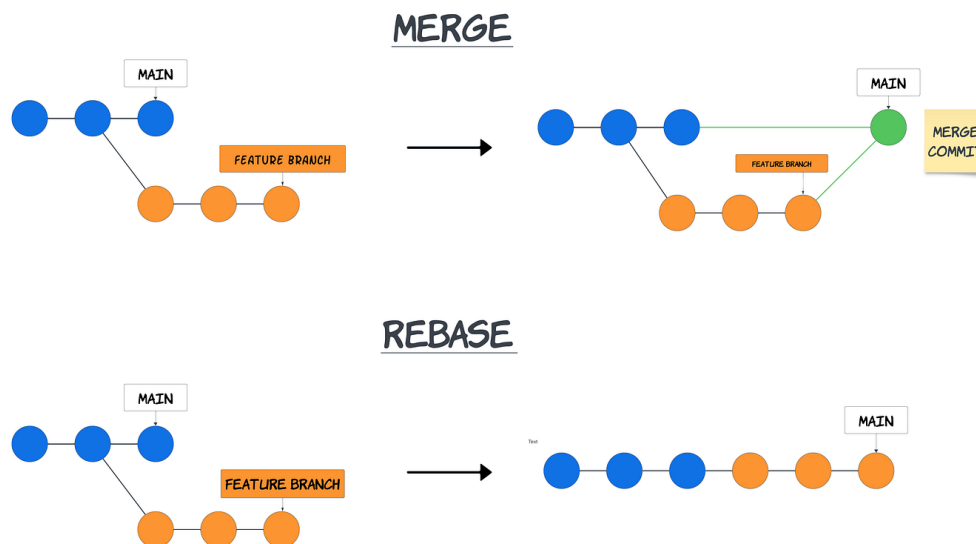


Activity #3 – Create a new branch

1. Click the “current branch” button and select “new branch”
2. Make changes to your file in the new branch and commit it
3. Switch back to the main branch and open the file, what do you notice?

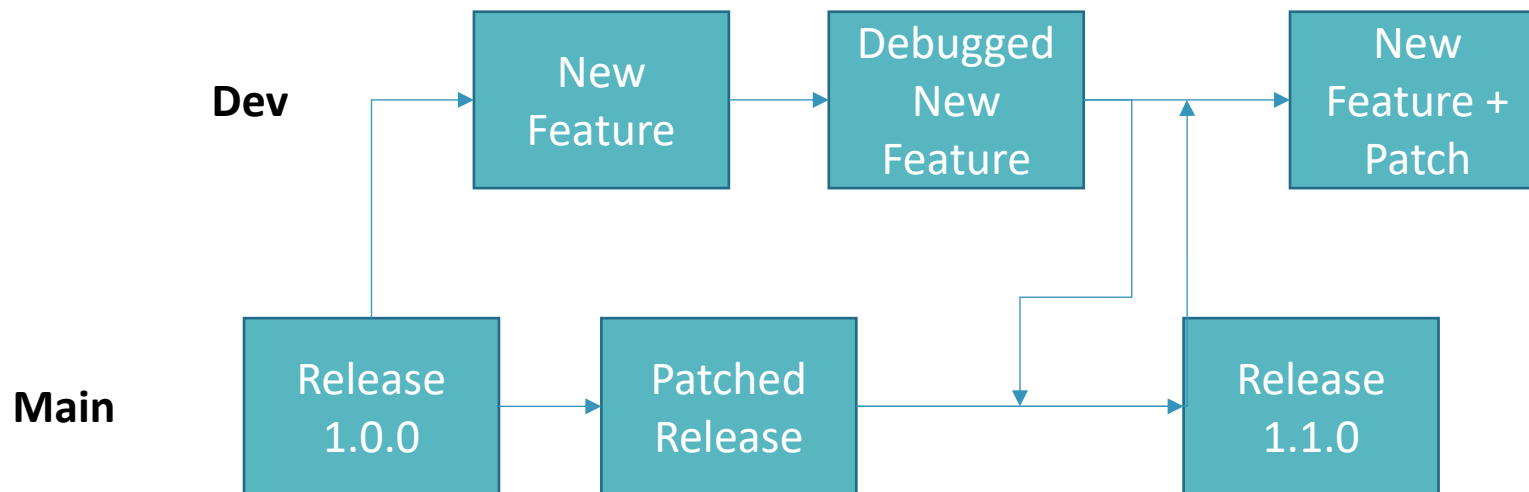
Now that your test feature works, you might want to make it part of your official release

Merge/Rebase



What is a Merge?

Action that will “merge”
the commits from one
branch into another



Activity #4.1 – Merging Branches



Activity #4.1 – Merging Branches

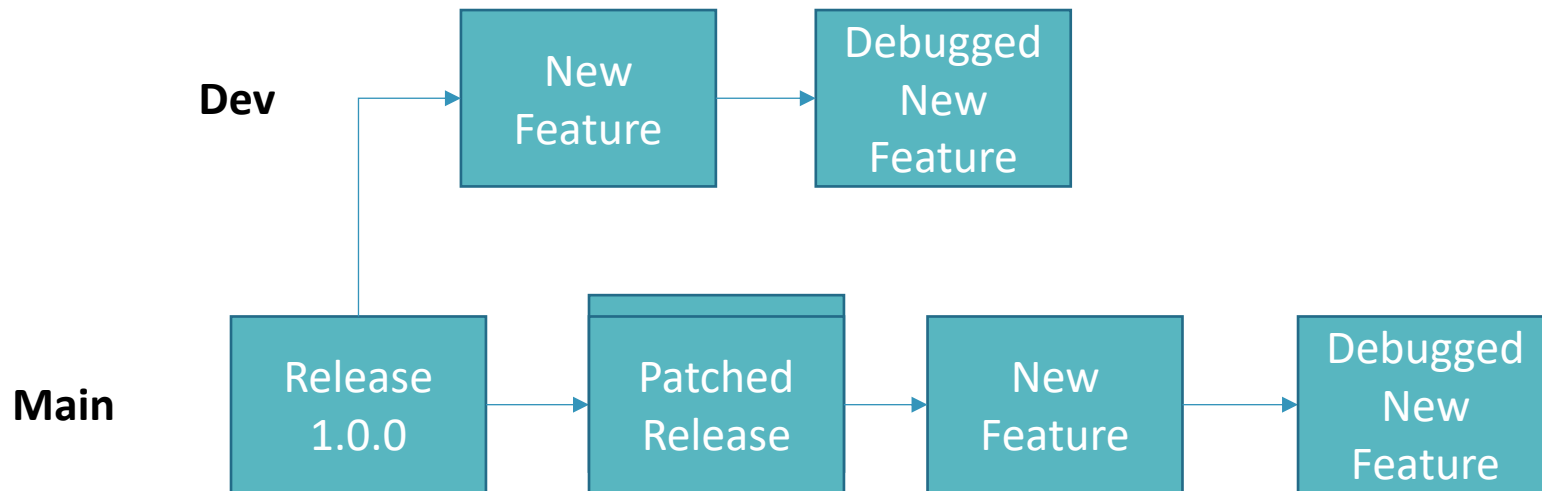
1. Switch to the main branch
2. Open the branch menu
3. Select the “choose a branch to merge into main” button and select your second branch.
4. Compare the branches again, the file should now be identical in both!



Rebasing

Rewrites the target branch's commit history to add other commits directly

Useful for updating dev branches with new main



Activity #4.2 – Rebasing Branches



Activity #4.2 – Rebasing

1. Create a new branch and commit some changes
2. Switch back to main and create a new change
3. Go back to the newly created branch
4. Open the branch menu and rebase off of main



Conflicts

If branches have competing commits, git will raise a **conflict** and not be able to merge/rebase

Can occur if:

- Both branches edit the same line
- Both added a file of the same name
- One branch edited a file, the other deleted it

Activity #5 – Resolving Conflicts



Activity #5 – Resolving Conflicts

1. In both branches, make a change on the **same line** and commit it
2. Try and merge your second branch into the main
3. When the merge conflict occurs, open the text file and resolve the issue
4. Try to merge again
5. Look at your commit history, how is your resolved merge shown in the commit?

Poll #3

What is the purpose of a merge/rebase?

T/F – Merging branches with changes in the same file will
ALWAYS create a merge conflict

How could we prevent conflicts from occurring?

Summary

- Branches can be used to develop new features without affecting a previous, viable version of your code
- Merges or Rebases can be used to combine the commits of two different branches
- Conflicts prevent branches from being combined



Part IV – Hosting Projects on GitHub



GitHub Interface



Why use remote repositories like GitHub?

- Backup for your code
 - Can access the repository from anywhere
- Collaborations
 - Easy to keep track of changes made by you and collaborators
- Open Science
 - Easy to access, download and use others' code
 - Easy to discuss with **users** of your code

In case of fire



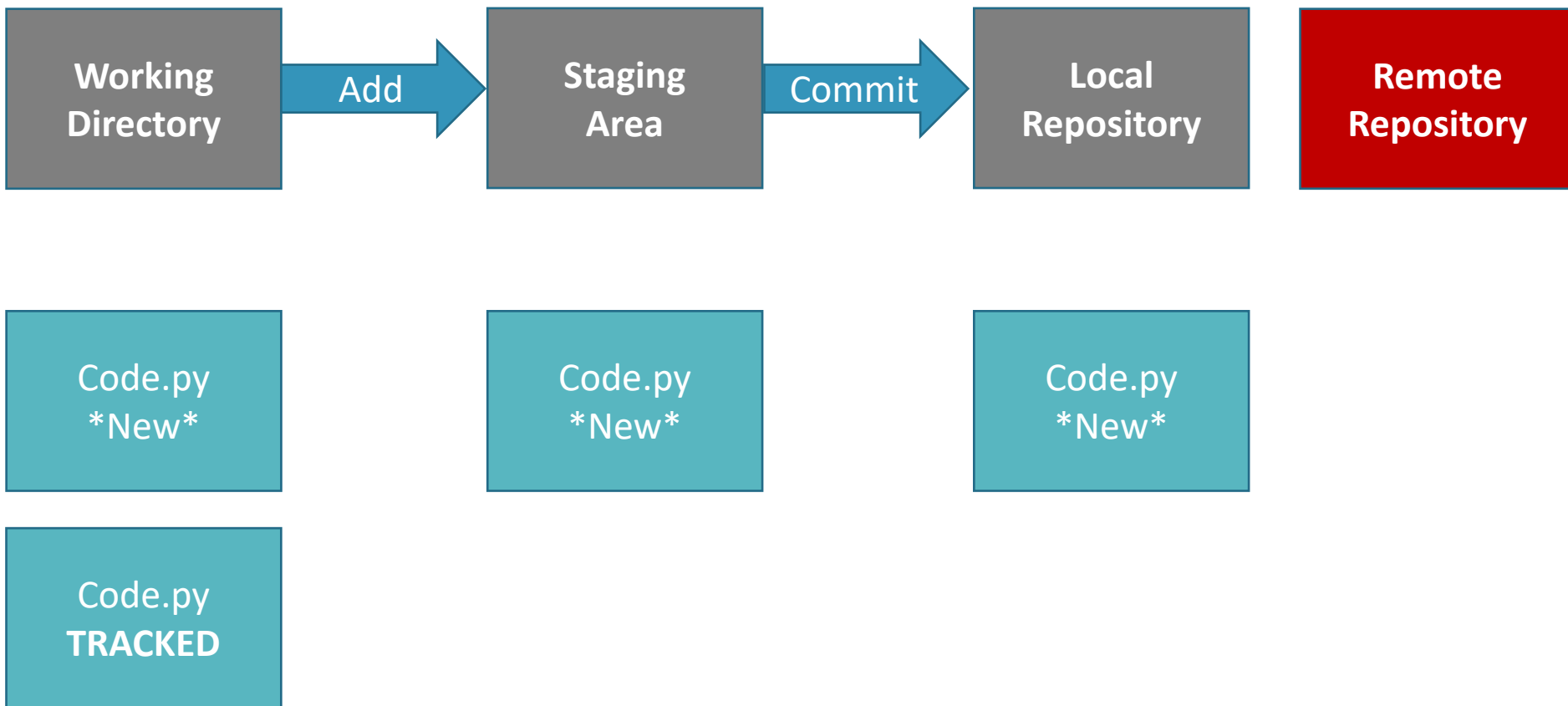
-  1. git commit
-  2. git push
-  3. leave building

Pushing and Pulling

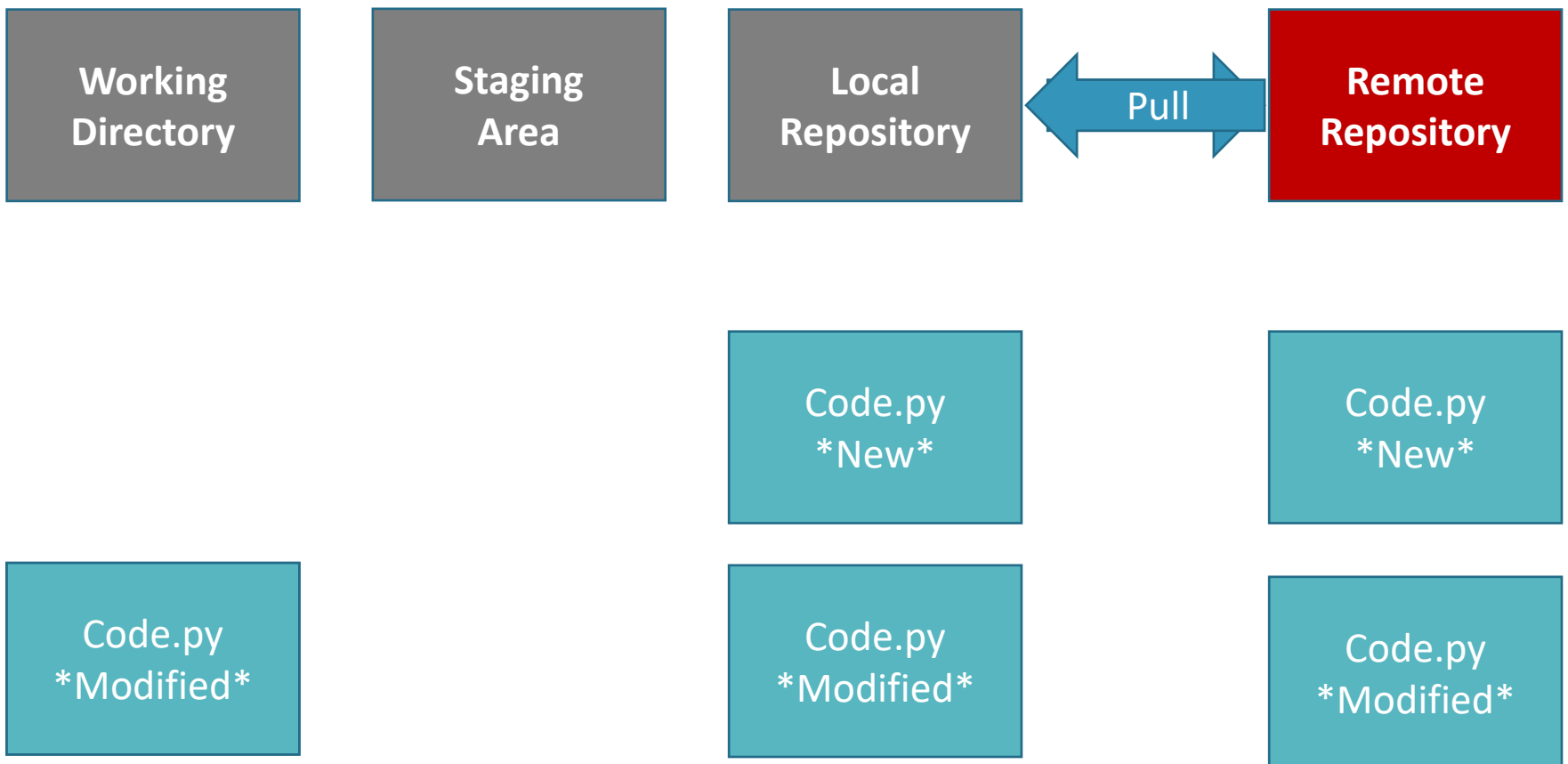
- **Push:** Updates your **remote** repository with recent commits from your **local** repository
- **Pull:** Updates your **local** repository with recent commits from your **remote repository**



Local-Remote Pipeline



Local-Remote Pipeline



Activity #6 – Pushing commits to GitHub



Activity #6 – Pushing commits to GitHub

1. Click the “Publish Repository” button
2. Now open your GitHub account in the browser and find your repository. See if your commit history is the same as in your local repo.

Activity #7 – Pulling Commits from GitHub



Activity #7 – Pulling Commits from GitHub

1. From the GitHub website, open your file and select the edit option
2. Make a change to the file and commit it.
3. Return to the desktop app and do Repository -> Pull
4. Look at your commit history and your file, what do you see?



Create a local copy of a GitHub Repository

- “Cloning”
- Usually done to **use** published code
 - Reproduce results
 - Use code on a dataset of interest
- Creates a **direct link** to the original repository
 - Anything changes you **push** will affect the original repository
 - Any changes in the original repo can be **pulled** into your clone

Cloning a Repository

Working
Directory

Local
Repository

Remote
Repository



Code.py
New

Code.py
New

Code.py
Modified

Code.py
Modified

Code.py
Modified



Activity #8 – Clone a Repository

- Find a GitHub repo that interests you and clone it!

Activity #8 – Clone a Repository

1. Go to the GitHub website and search for a repo that interests you.
2. Copy the repo's URL
3. In the Desktop App, do File → Clone a repository and paste the URL. (or on command line do ``git clone URL``)
4. Open the cloned repo and explore!

Alternatively can check my GitHub page to clone

Contributing to a Shared Repo

- Repo owner can add 'collaborators' with push privileges
- Collaborators can then push changes from their cloned copy of the repo
- ALWAYS make changes in a NEW BRANCH
 - Avoids merge conflicts and errors/bugs entering main branch

Activity #9.1 – Contribute to a shared repo

- Work in teams to contribute to a shared repository



1. Example Activities (Data Analysis // Database curation) are stored in 'Exercises' folder for this workshop's repo.
 - <https://github.com/ehuan2/IntroToGitHub>
2. **Task** – as a team, create a remote repo and collaborate to extend it using pull requests.
 - a) One user will create a base repo
 - b) The repo owner will add other members as repo collaborators
 - c) Make your changes in a **separate branch** and push the changes to the remote repository

Contributing to projects

- We saw earlier that we can create merge conflicts
- How can we regulate this when working in teams?
 - What if our 'team' can be anyone from the community?

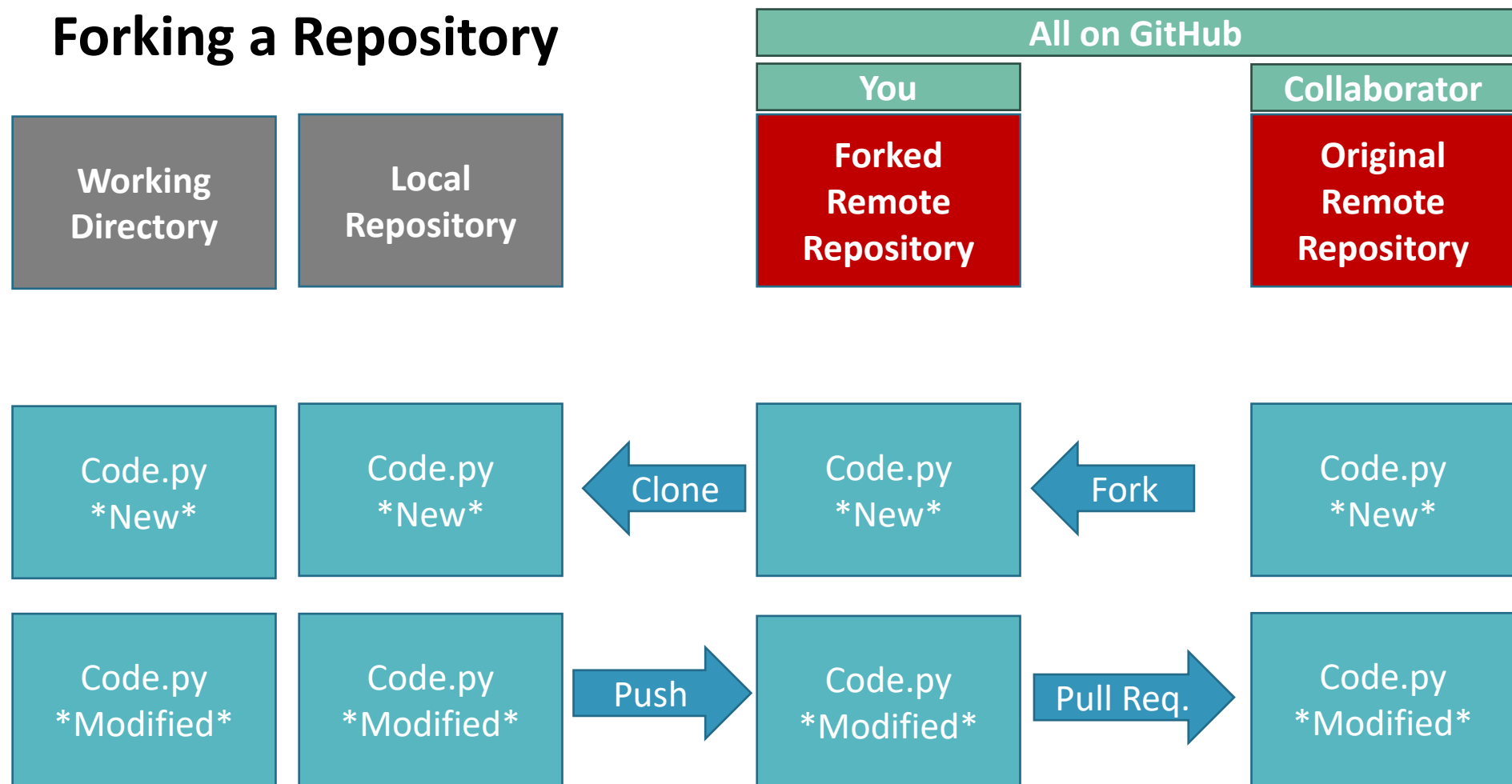
Forks & Pull Requests



Create a remote copy of a GitHub Repository

- “Fork”
- Usually used to **contribute** to published code
 - Can then edit the forked repository locally
 - Can then **push** local changes to your forked copy
 - When your changes are done, you can make a **pull request** to add your changes to the original repo
- Much safer way to contribute to source code

Forking a Repository



Activity #9.2 – Fork a Repo and make a Pull Request

- Fork the repo I made for this course and add a text file to it. Then, make a pull request!



Activity #9.2 – Fork a Repo and make a Pull Request

1. Open the link in chat to access the repo I made.
https://github.com/ehuan2/GitHub_Practice_Repo
2. Click the “fork” button
3. Clone your forked copy to your computer and add a text file with one thing you have learned today. Commit the file.
4. Push the commit to your forked repo.
5. On the GitHub website. Look at your forked repo and select the “create Pull Request” button

Poll #4

What command lets you update your remote repo with local changes?

T/F - Cloning a repository is the best way to contribute new features to someone else's repository

T/F – You will always be able to push changes from a cloned repo to the original

What is the advantage of using a Pull Request prior to merging changes from a contributor?



Summary

- GitHub provides a framework to store and document code
- Push/Pull commands allow you to transfer commits between local and remote repositories
- Forks/Pull Requests help manage code contributions



Important: Using GitHub is **NOT** a replacement for good communication:

- Ensures everyone is using the latest version of a file
- Helps keep track of what changes were made and by who

Good Practices:

- Have certain user(s) in charge of approving Pull Requests
- Set a standard format for commit messages
- Be descriptive in Pull Request comments

Part V – Miscellaneous Features



Using Git on the Command Line

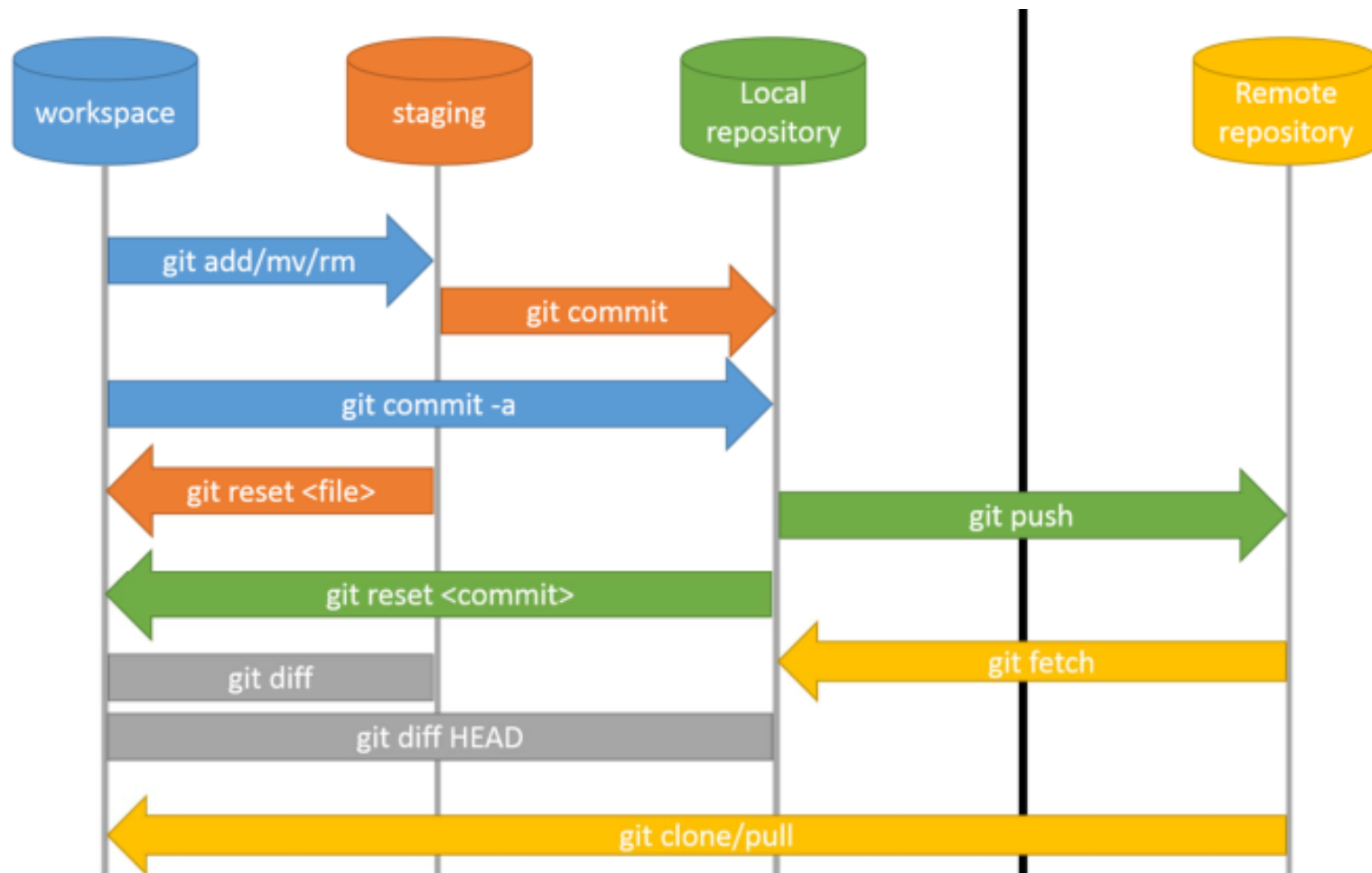


Using the Command Line:

- Direct interaction with Git
- Allows you to use Git on an HPC server (e.g: ComputeCanada)
- Provides more tools than third-party software

But:

- Can have a steep learning curve depending on experience with UNIX



Part VII – Conclusion



What have we learned?

- What VCS, Git, and GitHub are
- Why they are so useful
- How to undertake basic tasks in a local and remote repository
- How services like GitHub improve code collaboration and Open Science



What next?

- Practice makes perfect!
 - Track your hobby projects
 - Track your research
 - Use open-access code or software from publications
 - Consider trying Git on the command line



Workshop Feedback QR Code

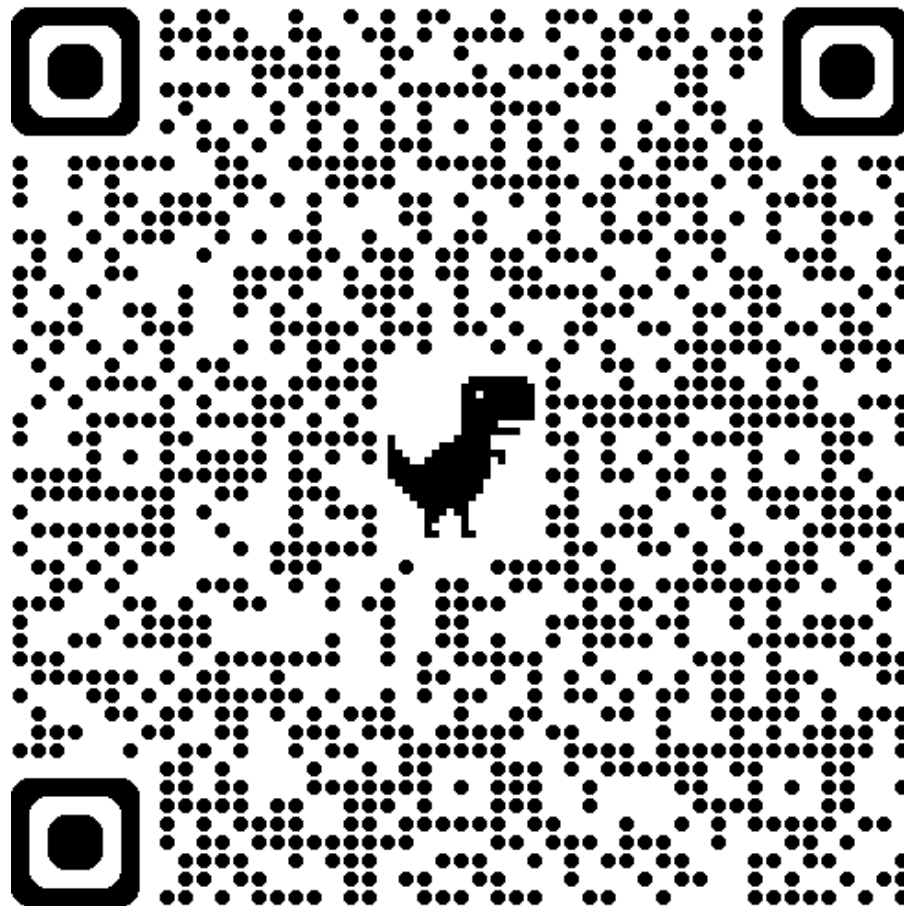


Image Sources

<https://betterexplained.com/articles/a-visual-guide-to-version-control/>

<https://github.com/torvalds>

<https://github.com/gitster>

<https://github.com/qw3rtman/git-fire>

<https://walkingrandomly.com/?p=6653>